

EXPERIMENT 3

Name: Arushi Gusain

Date: 27/08/2026

Subject: C programming

Batch: 15

Debugging Tools

1. Errors and Debugging

When we write a computer program, sometimes it does not work as expected. The mistakes present in a program are called **errors** (or bugs). The process of finding and correcting these errors is called **debugging**.

Definitions

- **Error:** A mistake or blunder made by the programmer in the source code. Example: Writing $a + b$ when the intention was to multiply $a * b$.
- **Bug:** A flaw or fault in a computer program that causes it to produce an incorrect or unexpected result, or to behave in unintended ways. Example: The program outputs the sum of two numbers instead of their product.
- **Debugging:** The systematic process of finding, analyzing, and removing bugs or defects in a computer program.

2. Classifications of Errors in C

Type of Error	Short Description	Cause	Example
Syntax Errors	Code violates the grammatical rules of the C language.	Missing semicolons, unmatched braces, typos in keywords.	int a = 10 (missing ;)
Compilation / Type Errors	Operations performed on incompatible types or missing declarations.	Assigning float string to int pointer, using undeclared variable.	int *p = "hello";
Linker Errors	The compiler generates object files, but the linker cannot combine them.	Function declared but not defined, missing libraries.	undefined reference to 'main'
Runtime Errors	The compiler generates object files, but the linker cannot combine them.	Division by zero, accessing invalid memory addresses.	int x = 10 / 0;
Logical Errors	Code runs to completion without crashing, but output is incorrect.	Wrong mathematical formula, incorrect loop condition.	for(i=0; i<5; i++) instead of <=5

CASE 1:

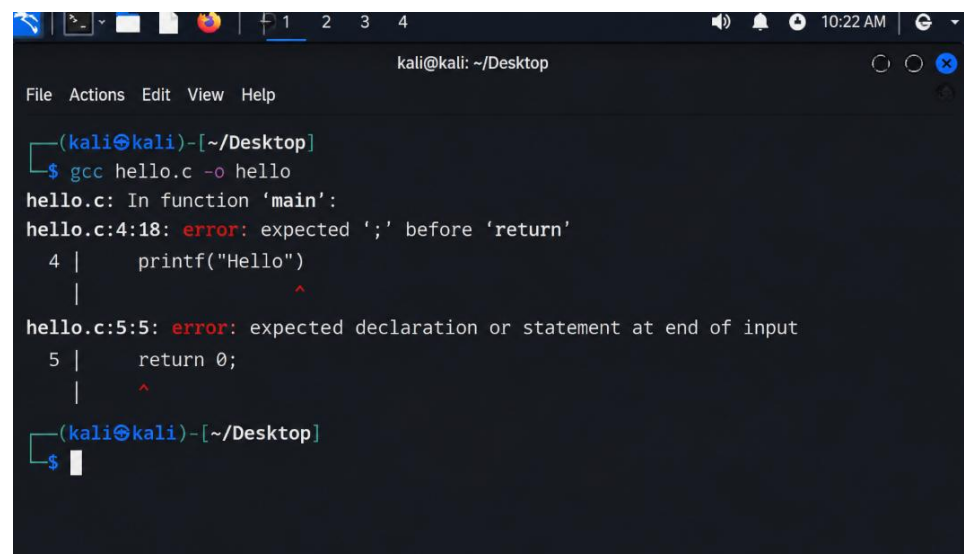
a) Syntax Error

Cause: Violation of C language grammar.

```
#include <stdio.h>

int main() {
printf("Hello")
return 0;
}
```

Error: Missing semicolon (;) after printf().



The screenshot shows a terminal window with the following content:

```
kali@kali: ~/Desktop
File Actions Edit View Help
(kali@kali)-[~/Desktop]
$ gcc hello.c -o hello
hello.c: In function 'main':
hello.c:4:18: error: expected ';' before 'return'
  4 |     printf("Hello")
    |                  ^
hello.c:5:5: error: expected declaration or statement at end of input
  5 |     return 0;
    |     ^
(kali@kali)-[~/Desktop]
$
```

(b) Compilation / Type Error

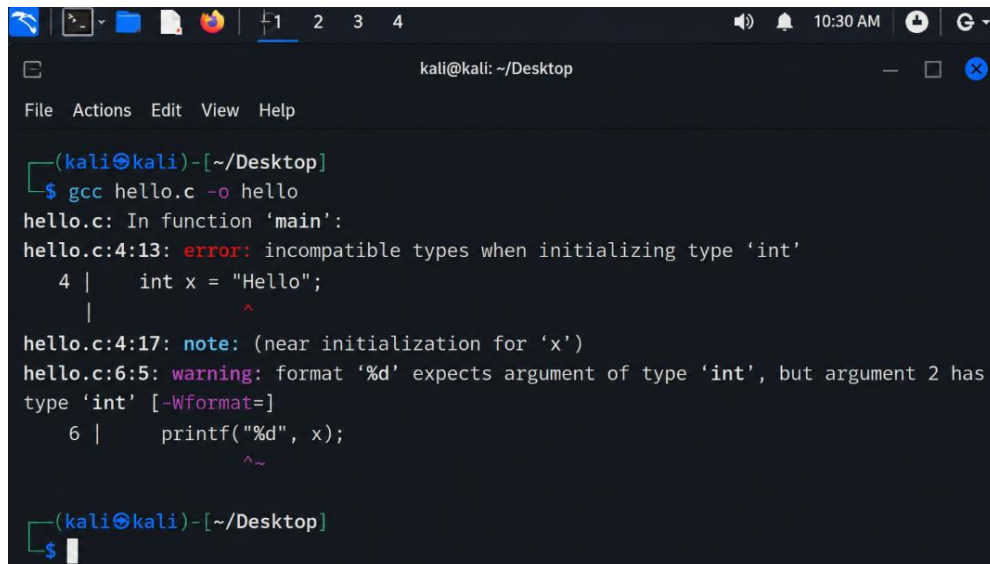
Cause: Using incompatible data types.

Code:

```
#include <stdio.h>

int main() {
    int x = "Hello";
    printf("%d", x);
    return 0;
}
```

Error: A string cannot be assigned to an integer variable.

A screenshot of a terminal window on a Kali Linux system. The window title is 'kali@kali: ~/Desktop'. The terminal shows the command 'gcc hello.c -o hello' being executed. The output displays two errors: 'hello.c: In function 'main': hello.c:4:13: error: incompatible types when initializing type 'int'...' and 'hello.c:6:5: warning: format '%d' expects argument of type 'int', but argument 2 has type 'int' [-Wformat=]'. The code being compiled is the same as shown in the previous block.

```
(kali@kali)-[~/Desktop]
$ gcc hello.c -o hello
hello.c: In function 'main':
hello.c:4:13: error: incompatible types when initializing type 'int'
   4 |     int x = "Hello";
     |           ^
hello.c:4:17: note: (near initialization for 'x')
hello.c:6:5: warning: format '%d' expects argument of type 'int', but argument 2 has
type 'int' [-Wformat=]
   6 |     printf("%d", x);
     |           ^~

(kali@kali)-[~/Desktop]
$
```

(c) Linker Error

Cause: Occurs when all compilation is successful, but linker cannot find the definition of a function or variable referenced in the code.

Code:

```
#include <stdio.h>

void myfunc(); // Declaration only

int main() {
    myfunc();
}
```

```
return 0;
}
```

Error: The program compiles, but the linker cannot find the definition of myfunc().

```
(kali@kali)-[~/linker_error]
$ gcc main.c
/usr/bin/ld: /tmp/ccr6m6oA.o: in function `main':
main.c:(.text+0x5): undefined reference to `myfunc'
collect2: error: ld returned 1 exit status
```

(d) Runtime Error

Cause: An illegal operation occurs while the program is running, such as division by zero.

Code:

```
#include <stdio.h>
int main() {
    int a = 10, b = 0;
    int c = a / b; // Division by zero
    printf("%d", c);
    return 0; }
```

Error: Program crashes during execution.

```
kali@kali:~/c_programs
$ gcc runtime_error.c -o runtime_error
kali@kali:~/c_programs
$ ./runtime_error
Floating point exception (core dumped)
kali@kali:~/c_programs
$
```

(e) Logical Error

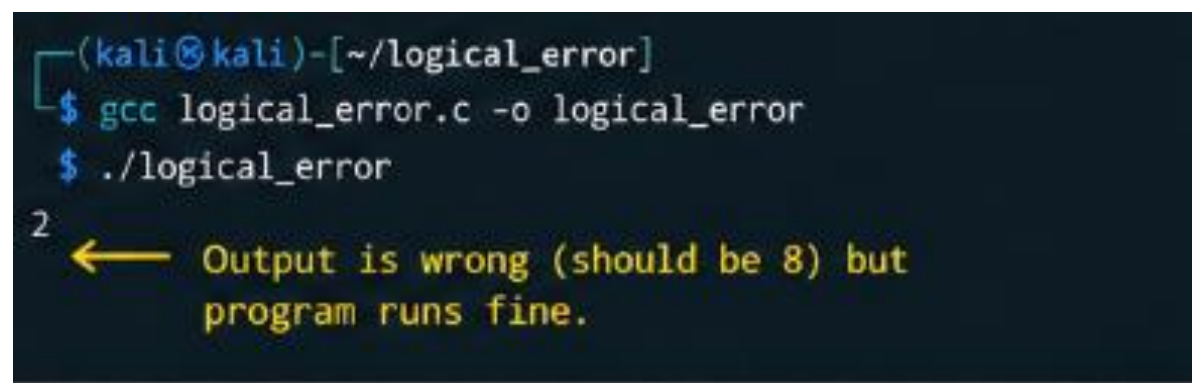
Cause: The program logic is incorrect, causing wrong output even though the program compiles and runs successfully.

Code:

```
#include <stdio.h>

int main() {
    int a = 5, b = 3;
    int c = a - b; // Should be a + b
    printf("%d", c);
    return 0;
}
```

Error: Logic is incorrect, but the program compiles and runs successfully.



```
(kali㉿kali)-[~/logical_error]
└─$ gcc logical_error.c -o logical_error
└─$ ./logical_error
2
    ← Output is wrong (should be 8) but
       program runs fine.
```

CASE 2: printf-Based Debugging (Logical bug)

Before using complex debugging tools, simple program flow and state inspection can be performed using output statements (printf).

```
(kali㉿kali)-[~/demo1]
└─$ more error3.c
#include <stdio.h>
int main()
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: i = %d\n" , i);
        sum = sum + i;
        printf("DEBUG: sum = %d\n" , sum);
    }
    printf("Final Sum = %d\n" , sum);
    return 0;
}

(kali㉿kali)-[~/demo1]
└─$ cc error3.c

(kali㉿kali)-[~/demo1]
└─$ ./a.out
DEBUG: i = 0
DEBUG: sum = 0
DEBUG: i = 1
DEBUG: sum = 1
DEBUG: i = 2
DEBUG: sum = 3
DEBUG: i = 3
DEBUG: sum = 6
DEBUG: i = 4
DEBUG: sum = 10
```

3. Introduction to GDB (GNU Debugger)

GDB stands for GNU Debugger.

The **GNU Project** is a free software movement started by **Richard Stallman** in **1983**. Its goal is to develop a complete **free and open-source Unix-like operating system** and software tools that users can run, study, modify, and share freely.

GNU stands for: **GNU's Not Unix**

Although GNU is similar to Unix, it is completely free software and was developed independently.

Some popular GNU software includes:

- GCC (GNU Compiler Collection)
- GDB (GNU Debugger)
- Bash (GNU Bourne Again Shell)
- Core Utilities (ls, cp, mv, etc.)

How GDB Fits into the GNU Ecosystem

GDB (GNU Debugger) is a debugging tool developed as part of the GNU Project.

It helps programmers:

- Find and fix errors (bugs) in programs
- Execute programs step by step
- View variable values during execution
- Set breakpoints
- Analyze program crashes

GDB works closely with **GCC**, the GNU compiler:

1. Write a program.
2. Compile it using GCC with debugging information (-g option).
3. Use GDB to debug the compiled program.

Example:

```
gcc -g program.c -o program
```

```
gdb program
```

4. Debugging Logical Errors with GDB

Logical errors can be difficult because the program may run normally. A good method is to stop near the suspicious statement, inspect the current values, and then move through the code one line at a time. For example, if a loop should calculate $1+2+3+4+5$ but gives 10, checking the loop counter can reveal that the final iteration is being skipped.

```

Reading symbols from ./error5...
(gdb) list
4      int result1;
5      result1 = x * x;
6      return result1;
7
8      int main
9      {
10         int result, n = 5;
11         result = square(n);
12         printf("%d\n", result);
13         return 0;
14     }
(gdb) break main
Breakpoint 1 at 0x1156: file error5.c, line 9.
(gdb) run
Starting program: /home/kali/demol/error5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-l
linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at error5.c:9
9      {
10         int result, n = 5;
(gdb) next
10         result = square(n);
(gdb) next
11         printf("%d\n", result);
(gdb) print result
$1 = 25
(gdb)

```

Installation & Compilation for GDB

Update package list and install GDB on Linux systems

\$ sudo apt update

\$ sudo apt install gdb

Compile source code with debugging symbols enabled (-g)

\$ cc -g -Wall -Wextra program.c -o program

Launch GDB with the compiled binary

\$ gdb ./program

Quick Reference: Common GDB Commands

Command	Shorthand	Description
Break<line/ func>	b	Sets a breakpoint at a specified line number or function name.
run	r	Starts execution of the program from the beginning.
next	n	Executes the next line of code (steps over function calls).
step	s	Executes the next line of code (steps into function calls).
print <var>	p	Prints the current value of a variable or expression.
display<var>	disp	Automatically prints the value of a variable after every step.
continue	c	Resumes execution until the next breakpoint or program completion.
list	l	Prints source code lines around the current execution position.
info locals	info loc	Lists all local variables and their current values in the current frame.

backtrace	bt	Displays the current call stack (chain of function calls).
info breakpoints	i b	Displays all active breakpoints and their numbers.
delete<id>	d	Deletes the specified breakpoint number (or all if omitted).
disable<id>	dis	Temporarily disables a breakpoint without deleting it.
enable<id>	en	Re-enables a previously disabled breakpoint.
quit	q	Exits the GDB debugging session.

Case 4: Debugging a Logical Error using GDB

Example: Code with Logic Bug (sample4.c)

```

#include                // Line 1
int main()              // Line 2
{                       // Line 3
int i, sum = 0;         // Line 4
for(i = 1; i < 5; i++)   // Line 5
(Logical Bug: should be i <= 5) { // Line 6
sum = sum + i;          // Line 7
}                       // Line 8
printf("Sum = %d\n", sum); // Line 9
return 0;               // Line 10

```

```

kali@kali: ~/gdb_test
File Actions Edit View Help

kali@kali:~/gdb_test$ gcc -g sample4.c -o sample4
kali@kali:~/gdb_test$ gdb sample4
GNU gdb (Debian 13.1-3) 13.1
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from sample4...
(gdb) break 5
Breakpoint 1 at 0x1149: file sample4.c, line 5.
(gdb) run
Starting program: /home/kali/gdb_test/sample4

Breakpoint 1, main () at sample4.c:5
5   for(i = 1; i < 5; i++)
(gdb) print i
$1 = 1
(gdb) print sum
$2 = 0
(gdb) next
6   {
(gdb) next
7   sum = sum + i;
(gdb) next
8   for(i = 1; i < 5; i++)
(gdb) print i
$3 = 2
(gdb) print sum
$4 = 1
(gdb) continue
Continuing.
Sum = 10
Program exited normally.
(gdb)

```

- Expected Output: 15 (1+2+3+4+5)
- Actual Output: 10 (1+2+3+4)

Case 5: Debugging Functions (next vs. step)

- **next**: Executes the line. If the line contains a function call, the function executes entirely in the background without entering its body.
- **step**: Steps into the function call, transferring GDB's focus into the function body for line-by-line inspection.

```
Breakpoint 1, main () at error5.c:9
9      {int result, n = 5;
(gdb) next
10      result = square(n);
(gdb) step
square (x=5) at error5.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6      return result1;
(gdb) print result
✗ No symbol "result" in current context.
(gdb) print result1
$2 = 25
(gdb) █
```

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.

Example code:

```
#include<stdio.h>
int divide(int a, int b)
{
    return a / b;          // Division by zero when b = 0
}
int calculate(int x) {
    return divide(x, 0);    // Passes 0 as divisor
}
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;              }
```

```
kali@kali: ~/gdb_demo
File Actions Edit View Help
kali@kali:~/gdb_demo$ gcc -g sample6.c -o sample6
kali@kali:~/gdb_demo$ gdb ./sample6
GNU gdb (Debian 13.1-3) 13.1
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample6...
(gdb) break divide
Breakpoint 1 at 0x1149: file sample6.c, line 4.
(gdb) run
Starting program: /home/kali/gdb_demo/sample6

Breakpoint 1, divide (a=10, b=0) at sample6.c:4
4      return a / b;           // Division by zero when b = 0
(gdb) print a
$1 = 10
(gdb) print b
$2 = 0
(gdb) next
Program received signal SIGFPE, Arithmetic exception.
0x00005655555514a in divide (a=10, b=0) at sample6.c:4
4      return a / b;           // Division by zero when b = 0
(gdb) backtrace
#0  0x00005655555514a in divide (a=10, b=0) at sample6.c:4
#1  0x000056555555166 in calculate (x=10) at sample6.c:9
#2  0x00005655555518b in main () at sample6.c:14
(gdb)

(gdb) list
1  #include <stdio.h>
2
3  int divide(int a, int b)
4  {
5      return a / b;           // Division by zero when b = 0
6  }
7
8  int calculate(int x)
9  {
10     return divide(x, 0);     // Passes 0 as divisor
11 }
12
13 int main()
14 {
15     int result = calculate(10);
16     printf("Result: %d\n", result);
17     return 0;
18 }

(gdb) bt
#0  0x00005655555514a in divide (a=10, b=0) at sample6.c:4
#1  0x000056555555166 in calculate (x=10) at sample6.c:9
#2  0x00005655555518b in main () at sample6.c:14

(gdb) info locals
a = 10
b = 0

(gdb)
```

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results in undefined behavior or memory corruption.

Example Code

```
#include <stdio.h>

int main() {
    int a[5];
    int i;
    for(i = 0; i <= 5; i++) {
        scanf("%d", &a[i]); //Line 8
    }
    return 0;
}
```

```

kali@kali: ~/gdb_demo
File Actions Edit View Help
kali@kali:~/gdb_demo$ gcc -g sample7.c -o sample7
kali@kali:~/gdb_demo$ gdb ./sample7
GNU gdb (Debian 13.1-3) 13.1
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample7...
(gdb) break 8
Breakpoint 1 at 0x114d: file sample7.c, line 8.
(gdb) run
Starting program: /home/kali/gdb_demo/sample7

Breakpoint 1, main () at sample7.c:8
8      scanf("%d", &a[i]);    //Line 8
(gdb) info locals
a = {0, 0, 0, 0, 0}
i = 0
(gdb) print i
$1 = 0
(gdb) next
Breakpoint 1, main () at sample7.c:8
8      scanf("%d", &a[i]);    //Line 8
(gdb) print i
$2 = 1
(gdb) continue
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x000056555555514d in main () at sample7.c:8
(gdb)

(gdb) list
1      #include <stdio.h>
2
3      int main()
4      {
5          int a[5];
6          int i;
7
8      →  for(i = 0; i <= 5; i++)
9          {
10             scanf("%d", &a[i]);    //Line 8
11         }
12
13         return 0;
14     }

(gdb) info breakpoints
Num   Type             Disp  Enb Address            What
1     breakpoint       keep  y   0x000056555555514d in main
      at sample7.c:8

(gdb) backtrace
#0  0x000056555555514d in main () at sample7.c:8
=> 8      scanf("%d", &a[i]);    //Line 8
(gdb)

```

Case 8: Segmentation Faults (SIGSEGV) using GDB

A Segmentation Fault occurs when a program attempts to access a memory location that it does not have permission to access (such as dereferencing a NULL pointer).

Example Code

```

#include <stdio.h>

int main() {
    int *p = NULL;    // Pointer points to address
    0x0 *p = 10; // Attempting to write to address 0x0 causes Segmentation
    Fault
    printf("%d\n", *p);
    return 0;
}

```

```
kali@kali: ~/gdb_demo
File Actions Edit View Help

kali@kali:~/gdb_demo$ gcc -g sample8.c -o sample8
kali@kali:~/gdb_demo$ gdb ./sample8
GNU gdb (Debian 13.1-3) 13.1
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sample8...
(gdb) break 5
Breakpoint 1 at 0x114d: file sample8.c, line 5.
(gdb) run
Starting program: /home/kali/gdb_demo/sample8

Breakpoint 1, main () at sample8.c:5
5      *p = 10; // Attempting to write to address 0x0
(gdb) info locals
p = 0x0
(gdb) next
6      printf("%d\n", *p);
(gdb) next

Program received signal SIGSEGV, Segmentation fault.
0x0000555555551156 in main () at sample8.c:6
6      printf("%d\n", *p);
(gdb) info locals
p = 0x0
(gdb) backtrace
#0 0x0000555555551156 in main () at sample8.c:6
(gdb) []

(gdb) list
1      #include <stdio.h>
2
3      int main()
4      {
5  ->     int *p = NULL; // Pointer points to address 0x0
6         *p = 10; // Attempting to write to address 0x0
7         printf("%d\n", *p);
8         return 0;
9     }
```

Shell Context

Shell:

A Shell is a command-line interface that allows the user to communicate with the operating system. It accepts commands from the user, interprets them, and executes them through the operating system.

Kernel:

The Kernel is the core part of an operating system. It manages hardware resources such as the CPU, memory, and input/output devices, and provides communication between software and hardware.

Three Common Linux Shells

1. **Bash (Bourne Again Shell)** – The most commonly used Linux shell and the default shell on many Linux distributions.
2. **Zsh (Z Shell)** – An advanced shell with features such as improved auto-completion, customization, and plugins.
3. **Fish (Friendly Interactive Shell)** – A user-friendly shell with syntax highlighting, auto-suggestions, and easy configuration.

In simple terms:

User → Shell → Kernel → Hardware

The shell takes commands from the user, while the kernel communicates with and manages the computer's hardware.

Difference Between Breakpoint, Watchpoint, Catchpoint, and Tracepoint

These are **debugging features in GDB (GNU Debugger)** used to control and observe the execution of a program.

1. Breakpoint

A **breakpoint** tells GDB to pause program execution when it reaches a particular line, function, or address.

Example:

```
(gdb) break 10
```

```
(gdb) run
```

The program stops when it reaches **line 10**.

2. Watchpoint

A **watchpoint** monitors a variable and pauses execution when its value changes.

Example:

```
(gdb) watch x
```

```
(gdb) run
```

GDB stops whenever the value of x changes.

3. Catchpoint

A **catchpoint** stops execution when a particular **event** occurs rather than at a specific line.

Example:

(gdb) catch throw

This stops when a C++ exception is thrown.

Other examples include:

catch syscall

catch signal

4. Tracepoint

A **tracepoint** is used to collect information about program execution while allowing the program to continue running. It is particularly useful for analyzing programs where repeatedly stopping execution would be inconvenient.

Example:

(gdb) trace main

Quick Comparison

- **Breakpoint** → "Stop here."
- **Watchpoint** → "Stop when this variable changes."
- **Catchpoint** → "Stop when this event happens."
- **Tracepoint** → "Record what happens here and continue."

Conclusion: Breakpoints focus on **locations**, watchpoints on **data changes**, catchpoints on **events**, and tracepoints on **collecting execution information without ordinary stopping**.
